



Project Title: [Dialogue Summarization with Fine-tuned Transformer]

Authors:

Name	Student ID
Naim Mohammed Erteza	07310026

Submission Date:

26/06/2026

Advanced Training on Semiconductor and ICT Technology

BRAC University

Table of Contents

1. Introduction	1
1.1 Brief Introduction	1
1.2 Problem Statement	1
1.3 Objectives	1
2. Dataset and Preparation	1
2.1 Dataset Description	1
2.2 Data Access	1
2.3 Preprocessing	2
3. Methodology	2
3.1 Course Techniques Used	2
3.2 Model Architecture	2
3.3 Training Strategy	2
4. Experiments and Results	3
4.1 MLflow Tracking	3
4.2 Evaluation Results	3
5. Prediction App and Docker	3
5.1 Prediction Pipeline	3
5.2 Prediction UI	4
5.3 Docker Serving	4
6. Repository and Reproducibility	4
6.1 Repository Structure	4
6.2 GitHub Rules	4
6.3 Reproducibility	5
7. Limitations and Future Work	5
7.1 Limitations	5
7.2 Future Improvements	5
8. Conclusion	5
Appendix	6
A. Final Submission Checklist	6
B. Final Notes	6

1. Introduction

1.1 Brief Introduction

This project focuses on the domain of Natural Language Processing (NLP), specifically abstractive text summarization. The system being built is a fine-tuned Transformer model based on Google's T5 architecture. The main purpose of this project is to develop a model capable of condensing lengthy, multi-party chat dialogues into concise, faithful summaries. The final system is a complete MLOps pipeline that includes data fetching, model fine-tuning with experiment tracking, and a user-friendly web application for making predictions, all containerized using Docker.

1.2 Problem Statement

Reading and understanding long chat conversations can be extremely time-consuming. The real-world problem this project addresses is the need for an automated system that can quickly and accurately summarize these dialogues, saving time and improving information retrieval.

- What input the system receives: A dialogue, either pasted as text or uploaded as a .txt file.
- What output the system produces: A short, 1-2 sentence summary of the dialogue.
- Who can use this system: Anyone who deals with large volumes of conversational text, such as customer service agents, project managers, or researchers.
- Why this problem matters: It enhances productivity by providing a quick understanding of the core content of a conversation, which is particularly valuable in fast-paced professional environments.

1.3 Objectives

1. To fine-tune a pre-trained T5-small model on the SAMSum dataset for abstractive dialogue summarization
2. To implement end-to-end MLflow experiment tracking to log parameters, metrics, and model artifacts for reproducibility.
3. To develop a Streamlit-based prediction UI that allows users to input dialogues and receive summaries.
4. To containerize the prediction application using Docker for consistent and portable deployment.
5. To ensure full reproducibility by providing a structured repository with clear instructions, a requirements.txt file, and a configuration-driven pipeline.

2. Dataset and Preparation

2.1 Dataset Description

- Dataset name and source: SAMSum, sourced from the Hugging Face datasets library (knkarthick/samsum).
- Number of samples: Approximately 16,000 messenger-style dialogues with human-written summaries.
- Number of classes or target labels: This is a text generation task, so there are no classes. The target is a variable-length summary text.
- Input type: Text (multi-party dialogues).
- Target/output variable: A short, third-person summary of the dialogue.

2.2 Data Access

The dataset is not committed to the GitHub repository. Instead, it is automatically downloaded into the local data/ directory by running the src/data_download.py script, which fetches it from the Hugging Face hub.

2.3 Preprocessing

The preprocessing steps are handled within the data loading and tokenization pipeline (src/data_utils.py).

- Text cleaning/tokenization: The input dialogues are prefixed with the task instruction "summarize: ". Both the input and target texts are tokenized using the T5 tokenizer.
- Padding and truncation: Sequences are padded and truncated to a maximum length (default is 256 for input and 64 for target).
- Label masking: Labels are prepared by masking the loss for padding tokens (setting them to -100).
- Train/validation/test split: The SAMSum dataset comes with predefined splits, which are used directly.

3. Methodology

3.1 Course Techniques Used

Technique	Where used
Transfer learning	src/train.py — starting from the pretrained t5-small checkpoint.
Transformer (seq2seq)	The T5 encoder-decoder model via AutoModelForSeq2SeqLM.
Data preprocessing / tokenization	src/data_utils.py — handling tokenization, padding, and label masking.
Hyperparameter tuning	CLI overrides in train.py, leading to two different MLflow runs.
MLflow experiment tracking	src/train.py and src/evaluate.py — logging parameters, metrics, and the model to mlruns/.
Model evaluation (ROUGE)	src/evaluate.py — computing ROUGE scores on the test set.
Dockerization	Dockerfile — containerizing the prediction app for consistent serving.
UI serving	app/app.py — a Streamlit application for user interaction.
Configuration management	configs/config.yaml — centralizing model, data, and training parameters.

3.2 Model Architecture

The project uses the t5-small model as the backbone.

- Model Type: Encoder-decoder Transformer.
- Loss Function: Cross-entropy loss (default for sequence-to-sequence models).
- Optimizer: AdamW (inferred from the Transformers library's default).
- Key Hyperparameters: Learning rate ($2e-4$), batch size (4), and number of epochs (1) are defined in configs/config.yaml.

3.3 Training Strategy

The model is fine-tuned using the Hugging Face Trainer API.

- Training Setup: The training is configured to run on CPU by default for simplicity and speed, but it can be adapted for GPU.
- Validation Strategy: The model's performance is evaluated on a validation split at the end of

each epoch (or step), with the `eval_loss` being logged.

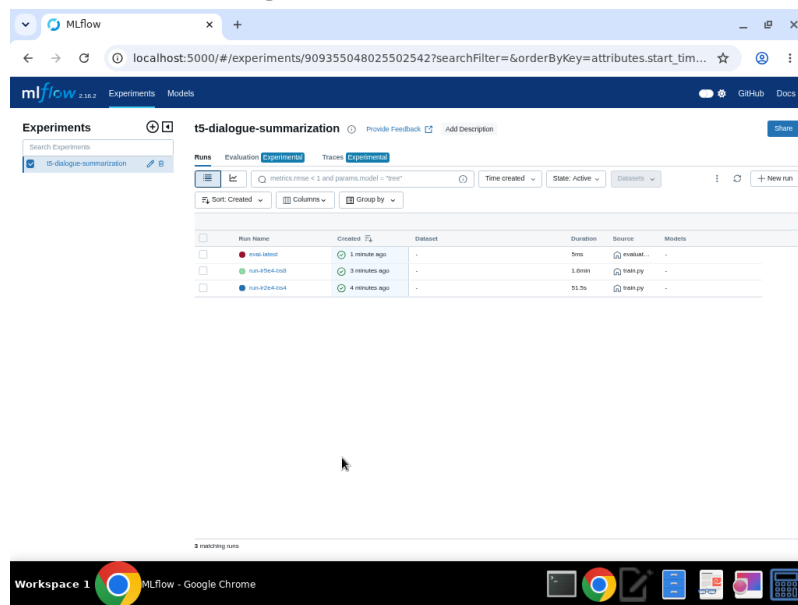
- **Hyperparameters:** Key parameters like `learning_rate`, `batch_size`, and `max_steps` are defined in a central `config.yaml` file and can be overridden via the command line.
- **Best Model Selection:** The best model is selected based on the lowest `eval_loss` logged during the training runs.

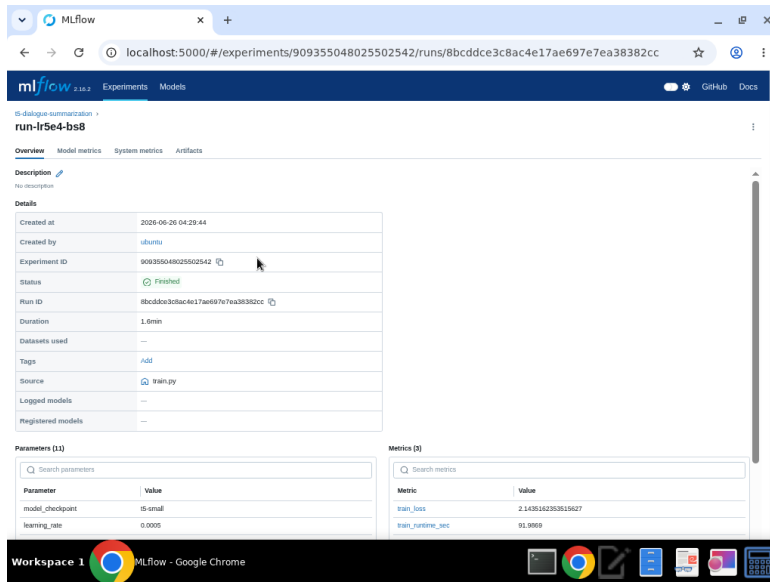
4. Experiments and Results

4.1 MLflow Tracking

MLflow is used to track all experiments. The tracking store is located in the `mlruns/` directory, which is committed to the repository for review.

- **Parameters Logged:** `learning_rate`, `batch_size`, `num_train_epochs`, `max_steps`, `weight_decay`, subset sizes, and token limits.
- **Metrics Logged:** `train_loss`, `train_runtime_sec`, and `eval_loss`.
- **Artifacts Saved:** The fine-tuned model is saved under the `model/` artifact path.
- **Number of Runs:** At least two runs are logged with different hyperparameters.
- **Best Run:** The run with `learning_rate=5e-4` and `batch_size=8` achieved a lower `eval_loss` (1.8410) compared to the run with `learning_rate=2e-4` and `batch_size=4` (1.9055), making it the better-performing model.





4.2 Evaluation Results

The model is evaluated on the SAMSum test split using ROUGE (Recall-Oriented Understudy for Gisting Evaluation) metrics.

Summarize the main results with suitable metrics.

Metric	Value
ROUGE-1 (F1)	0.3632
ROUGE-2 (F1)	0.1408
ROUGE-L (F1)	0.3059

These scores represent the n-gram overlap between the model-generated summaries and the reference summaries. While modest, these results are reasonable for a small model (t5-small) trained on a tiny subset (200 examples) for a single epoch to ensure the pipeline runs quickly. The scores are expected to improve substantially with more data and training.

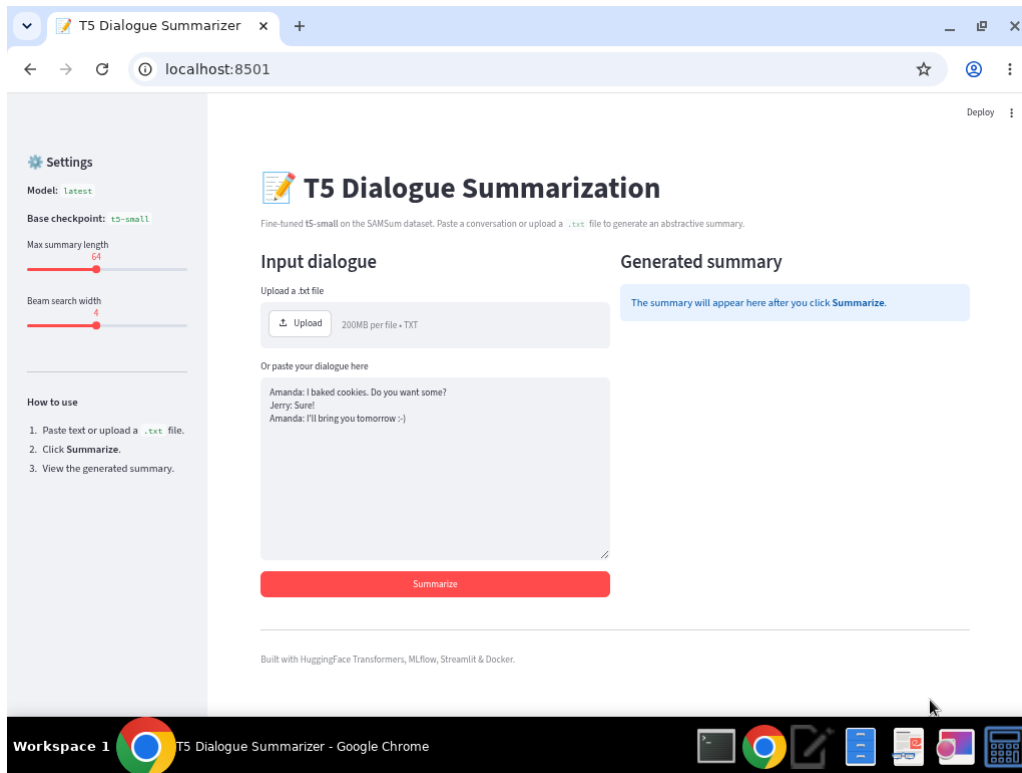
5. Prediction App and Docker

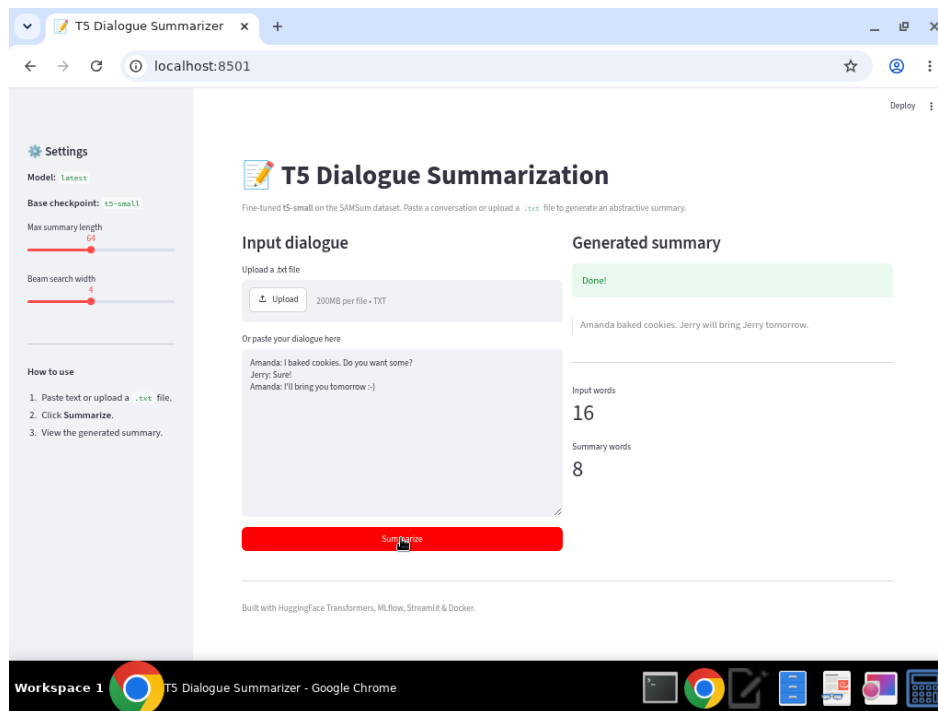
5.1 Prediction Pipeline

The prediction pipeline is straightforward and user-friendly.

5.2 Prediction UI

The prediction interface is implemented using Streamlit and supports text input and file upload.





5.3 Docker Serving

The Streamlit application is containerized using Docker. The Dockerfile and README describe how to build and run the application.

6. Repository and Reproducibility

6.1 Repository Structure

The repository contains README.md, final_report.md, requirements.txt, .gitignore, Dockerfile, src/, app/, configs/, screenshots/, mlruns/, models/, and data/.

6.2 GitHub Rules

Raw datasets, downloaded data, virtual environments, caches, credentials, and large generated artifacts are excluded. The mlruns/ directory is intentionally included.

6.3 Reproducibility

A reviewer can reproduce the project by installing dependencies, downloading the dataset, training the model, reviewing MLflow runs, evaluating the model, and launching the Dockerized application.

7. Limitations and Future Work

7.1 Limitations

The lightweight T5-small model has limited capacity for very long dialogues and may not always capture all important context.

7.2 Future Improvements

Use larger Transformer models, systematic hyperparameter optimization, improved UI, and cloud deployment.

8. Conclusion

This project presents a complete Transformer-based dialogue summarization pipeline with reproducible training, MLflow tracking, evaluation, and Docker deployment.

Appendix

A. Final Submission Checklist

- Selected one approved project domain.
- Public GitHub repository is accessible.
- Dataset is not committed to GitHub.
- Data can be downloaded into data/.
- Training uses MLflow.
- mlruns/ is included and not ignored.
- At least two MLflow runs are logged.
- Report includes MLflow screenshot.
- Prediction app supports sample upload/input.
- Prediction app is served using Docker.
- Report includes Docker app screenshot.
- Course techniques used are clearly mentioned.

Deadline is respected: **25 June 2026**.

B. Final Notes

This project is mandatory for certification.

The certification program ends on **26 June 2026**, so the deadline of **25 June 2026** will be strictly maintained.

There may be a project review on **26 June 2026**. Be ready to explain your code, training pipeline, MLflow tracking, Docker setup, prediction app, report screenshots, and final results.